UNIVERSITY OF LEEDS

COMP1850 Programming

Week 3 Session 1

Loops, repetition, iteration

UNIVERSITY OF LEEDS

Recap

Last week we covered

- Python's collection types (lists, tuples, sets, dictionaries)
- Booleans and Truthy/Falsey
- Selection statements (`if`, `if-else`, `if-elif-else`, `match`)

UNIVERSITY OF LEEDS

Today's Aims

To understand how we use repetition in programming

- Logical control of repetition: `while` loops
- Iteration over data structures: `for` loops
- Iteration and collections

UNIVERSITY OF LEEDS

Repetition / Iteration / Loops

- We use these terms interchangeably to refer to programming structures that repeat sets of instructions
- Key to efficient program design:
  - We may be processing large amounts of data with the same operations
    - We can implement this in an efficient way
  - We may need logical control over repetition
    - We can write more dynamic programs

UNIVERSITY OF LEEDS

Basic While loop

```
while logical_expression:
        Statements to execute while True
```

- The simplest form of the `while` loop repeats a set of statements "while" a logical expression is `True`
- When the logical expression becomes `False`, the repetition ends
- All statements inside the loop must have the same level of indentation

UNIVERSITY OF LEEDS

Task 1: while loop

- Go to `week1.3/session1/task1/`
- Read the instructions in README.md
- Complete the exercise

UNIVERSITY OF LEEDS

While loops – risks

```
while logical_expression:
        Statements to execute while True
```

- What happen when the logical expression never becomes `False`?
  - We have created an infinite loop!
  - A common programming error
- We have to design our programs to avoid this trap

While loops – risks

We either have to:

- Be fully confident that it will terminate
- Allow an escape route – termed **`break`**

UNIVERSITY OF LEEDS

While loops – break

```
while logical_expression:
    Statements to execute while True
    if break_condition:
        break
```

We introduce a second logical expression using `if`:

- We must be sure *break_condition* will become `True`
- The **`break`** statement immediately exits the loop

UNIVERSITY OF LEEDS

Task 2: while-break

- Go to `week1.3/session1/task2/`
- Read the instructions on README.md
- Complete the exercise

While loops – continue

- Our program logic sometimes requires us jump over the statements but continue the loop
- We could construct equivalent logic with `if-else` but we have a more elegant solution: **`continue`**

UNIVERSITY OF LEEDS

While loops – continue

```
while logical_expression:
    if continue_condition:
        continue
    Statements to execute while True
```

- We introduce a further logical condition using `if`
- **`continue`** will jump directly to the while statement
- But the loop will 'continue' - compared to `break`

While loops – exiting the loop

UNIVERSITY OF LEEDS

In some situations, there may be specific statements relating to the end of a while loop

- It is better programming style to emphasise this in your code

UNIVERSITY OF LEEDS

While loops – else

```
while logical_expression:
    Statements to execute while True
else:
    Statements to execute after the loop
```

- **else** is executed once the `while` loop ends
- Best practice is to only include statements related to the end of that loop
- `break` will also jump past the `else` statements

UNIVERSITY OF LEEDS

Task 3: while-else

- Go to `week1.3/session1/task1/`
- Read the instructions on README.md
- Complete the exercise

UNIVERSITY OF LEEDS

while structure and program logic

```
while condition1:
    if condition2:
        continue
    Statements
    if condition3:
        break
    else:
        Statements
```

- The full set of statements creates a complex logical structure
- We often use a smaller subset of the control statements as shown in the examples

Do...while

Languages other than Python often have a related looping structure called `do...while`

- The logical control is at the end of the loop not the start
- It is sometimes a better expression of the required logic

```
while condition:
    Statements
```

```
do:
    Statements
while condition
```

- You can see that this second form of loop does not fit the Python syntax

UNIVERSITY OF LEEDS

Do...while logic in Python

In Python we construct equivalent logic with `break`

```
while True:
    Statements to execute while True
    if logical_expression:
        break
```

- We construct an infinite loop
- But use the logical expression to end (`break`) the loop

UNIVERSITY OF LEEDS

For loops - iteration

For loops are a much more general programming structure in Python than in many other languages

- Rather than just counting through a range we can iterate directly through data structures
- The combination of for loops with lists and other structures allows concise syntax for complex tasks

UNIVERSITY OF LEEDS

For loops - counting

```
for k in range(4):
    Statements to execute
```

This is the Python equivalent for the traditional for loop construct in other programming languages

- The built-in `range()` function produces a sequence of integers:

  `0, 1, 2, 3`

- Note: We count from 0 up to (but not including) the limit

UNIVERSITY OF LEEDS

For loops - range( )

`range()` can be used more generally:

```
range(start, stop, step)
```

We count:

- from `start`
- up to (but not including) `stop`
- in increments of `step`

UNIVERSITY OF LEEDS

Task 4: for

- Go to `week1.3/session1/task4/`
- Read the instructions on README.md
- Complete the exercise

For loops - iteration

UNIVERSITY OF LEEDS

If we want to select items from a list a naïve approach could be

```
animals = ['dog', 'cat', 'hamster', 'goldfish']
for k in range(4):
    item = animals[k]
    print(item)
```

- We use the index of the list to extract items
- But Python offers a much better approach

UNIVERSITY OF LEEDS

For loops - iteration

For loops operate directly on data structures

```
animals = ['dog', 'cat', 'hamster', 'goldfish']
for item in animals:
    print(item)
```

- The **`in`** keyword extracts an item from the list
- **`for`** iterates over the data structure

UNIVERSITY OF LEEDS

For loops - strings

Because character strings can also be considered as ordered lists of characters, we can iterate through them

```
course = "Computer Science"
for ch in course:
    print(ch)
```

- ch extracts one character at each iteration

UNIVERSITY OF LEEDS

Task 5: for

- Go to `week1.3/session1/task5/`
- Read the instructions on README.md
- Complete the exercise

For loops - logical control

UNIVERSITY OF LEEDS

We can use

- `else` to execute statements when the for loop is complete
- `continue` to jump to the next iteration
- `break` to exit the for loop, including any else statements

with `for` loops exactly as with `while` loops.

Logical control of `for` loops is not as common, but is sometimes useful

Summary

We have

- Considered how we use repetition in programming
- Explored logical control of repetition, using while loops
  - `while`, `break`, `continue`, `else`
- Explored iteration using for loops
  - Simple iteration over ranges
  - Iteration over data structures (e.g., lists)